“시간이 아깝지 않은”

SK네트웍스 Family AI캠프 교재5

강사 이영배

데이터 분석

- 개념 : 데이터를 수집, 정제, 해석하여 숨겨진 패턴과 유용한 정보(인사이트)를 발견하고, 이를 바탕으로 미래의 발생 가능한 상황을 예측하고 최적의 의사결정을 내리는 일련의 과정
- 분석 과정
  1) 데이터 수집 : 분석에 필요한 데이터 확보
  2) 데이터 전처리(preprocessing) : 데이터를 분석에 적합한 형태로 만들어서 신뢰도를 높이는 작업
  3) 데이터 탐색 : 그래프나 통계적인 방법을 이용, 데이터의 특징과 패턴을 파악
  4) 모델링 : 수집・정제된 데이터를 기반으로 수학적・통계적인 방법을 사용하여 데이터 간의 관계를 파악하고, 새로운 데이터에 대해서 결과를 예측하거나 설명할 수 있는 규칙(Model)을 만드는 과정
  5) 분석 모델 평가

07. 파이썬 라이브러리

- numpy / pandas

numpy(넘파이)

- 정의: 수학적 계산을 위한 python 라이브러리

- 자료형: 다차원 배열(ndarray) ↔ 파이썬 리스트

```
[0 1 2 3 4 5 6 7 8 9]
```

numpy(넘파이)

- python list 자료형과 numpy 다차원 배열 자료 구조 비교

	python list	numpy 다차원 배열
존재 목적 / 장점	다양한 자료형의 데이터 혼합 저장: 정수, 실수, 문자열, 심지어 다른 리스트 등 어떤 종류의 데이터든 하나의 리스트 안에 자유롭게 담을 수 있음	• 단일 종류의 숫자 데이터 중심 • 대규모 수치 데이터를 묶어서 빠르고 효율적으로 처리
요소별 수학 연산	불가능	가능(최대 장점)

numpy(넘파이)

- np.array(data) 함수:
  - data = python 리스트 또는 튜플, python 리스트(튜플)로부터 생성

```
list_data = [1,2,3,4,5]
print(list_data)
```

```
[1, 2, 3, 4, 5]
```

```
arr = np.array([1,2,3,4,5])
print(arr)
```

```
[1 2 3 4 5]
```

```
arr = np.array(list_data)
print(arr)
```

```
[1 2 3 4 5]
```

- 사용하기 : 라이브러리 임포트

  import numpy as np

  → numpy 라이브러리를 import하고 앞으로 np라는 이름으로 부른다.

numpy(넘파이)

- 주요 특징

1) 압도적인 연산 성능과 효율성: 대량의 숫자 데이터를 매우 빠르게 처리할 수 있음

2) 풍부한 수학적 연산 기능 제공: 사칙연산, 통계(평균, 합계, 표준편차), 선형 대수, 푸리에 변환 등 데이터 분석에 필수적인 다양한 수학 함수들을 제공

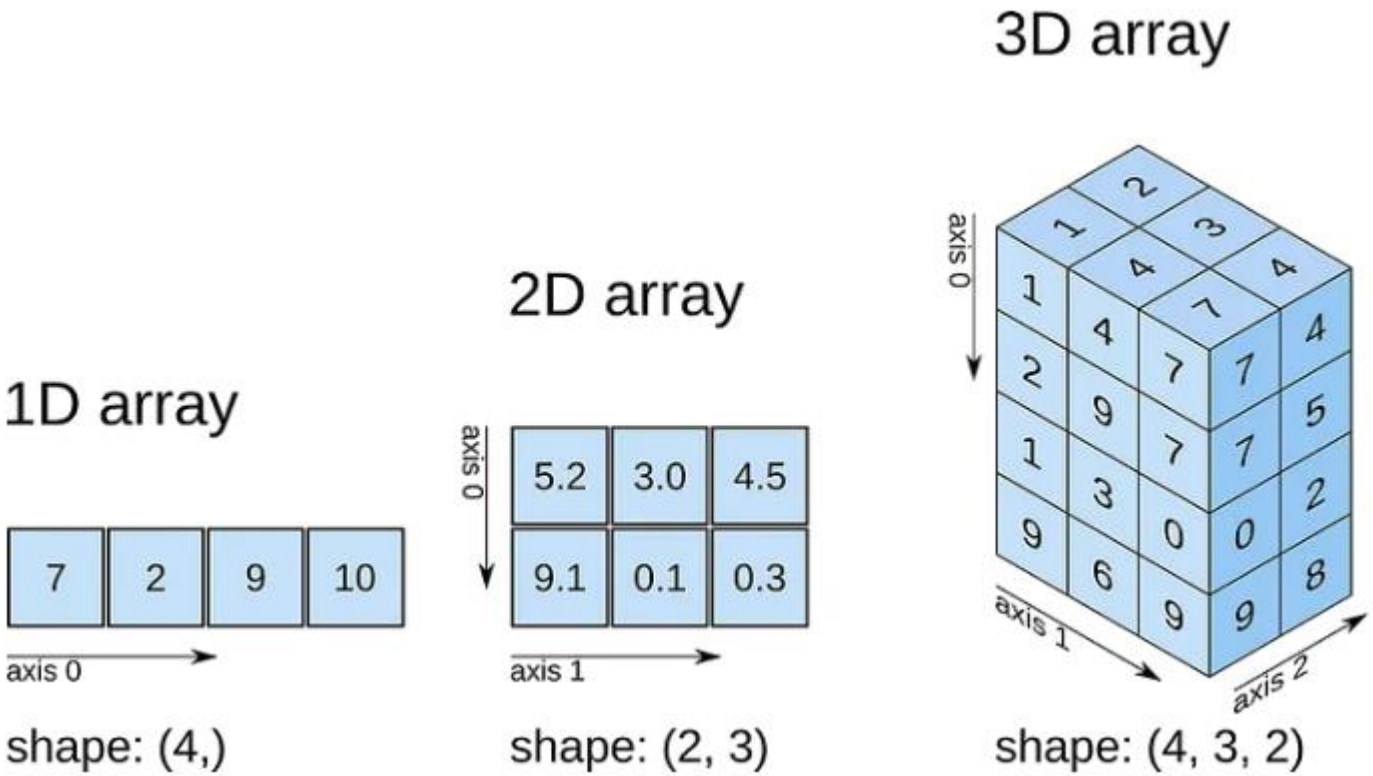
array 생성 하기

- 변수 = np.array(data)
- data = 대괄호가 중첩된 리스트

```
list_data = [[1,2,3],[4,5,6],[7,8,9]]
arr = np.array(list_data)
print(arr)
```

```
[[1 2 3]
 [4 5 6]
 [7 8 9]]
```

배열의 차원과 모양


1D array
7
2
9
10
axis 0
shape: (4,)
2D array
5.2
3.0
4.5
9.1
0.1
0.3
axis 0
axis 1
shape: (2, 3)
3D array
axis 0
axis 1
axis 2
shape: (4, 3, 2)

array 인덱싱

```
1 arr = np.array([[1,2,3],[4,5,6]])
2 arr
```

```
array([[1, 2, 3],
       [4, 5, 6]])
```

```
1 print(arr[0])
```

```
[1 2 3]
```

```
1 print(arr[0][0])
```

```
1
```

```
1 print(arr[0,0])
```

```
1
```

1차원 array 슬라이싱

배열[start 인덱스 : stop 인덱스]

- start 인덱스부터 stop 인덱스 바로 전까지 값 추출 (start <= x < stop)

```
# 1차원 배열 생성

## python range 함수 사용 -> 0~9 사이의 연속적인 숫자 데이터 생성
data = range(10)

## 넘파이 배열 생성
arr = np.array(data)
print(f'생성된 넘파이 배열: {arr}')
```

```
생성된 넘파이 배열: [0 1 2 3 4 5 6 7 8 9]
```

```
# 1차원 배열: 인덱스 3부터 7까지 슬라이싱
arr1 = arr[3:8]
print(f'슬라이싱 결과: {arr1}')
```

```
슬라이싱 결과: [3 4 5 6 7]
```

2차원 array 슬라이싱

배열[행 start 인덱스 : 행 stop 인덱스, 열 start 인덱스 : 열 stop 인덱스]

```
# 2차원 배열 생성

## python 리스트 + range 함수 사용 -> 0~39 사이의 연속적인 숫자 데이터 생성
list_data = [range(0,10),range(10,20),range(20,30),range(30,40)]

## 넘파이 배열 생성
arr = np.array(list_data)
print(f'생성된 넘파이 배열: ₩n{arr}')
```

```
생성된 넘파이 배열:
[[ 0  1  2  3  4  5  6  7  8  9]
 [10 11 12 13 14 15 16 17 18 19]
 [20 21 22 23 24 25 26 27 28 29]
 [30 31 32 33 34 35 36 37 38 39]]
```

```
# 2차원 배열: 행 인덱스 전체, 열 인덱스 4부터 9까지 슬라이싱
arr1 = arr[:,4:9]
print(f'슬라이싱 결과: ₩n{arr1}')
```

pandas

1. 정의

: 행(Row)과 열(Column)로 구성된 표 형식의 데이터(Tabular Data)를 효율적으로 다루기 위해 설계된 라이브러리

2. 주요 특징

: 다양한 형식의 정형 데이터 처리: CSV, Excel 파일, DB 테이블 등 다양한 형태의 정형 데이터를 파이썬에서 쉽고 유연하게 불러오고, 정제하며, 분석할 수 있도록 지원

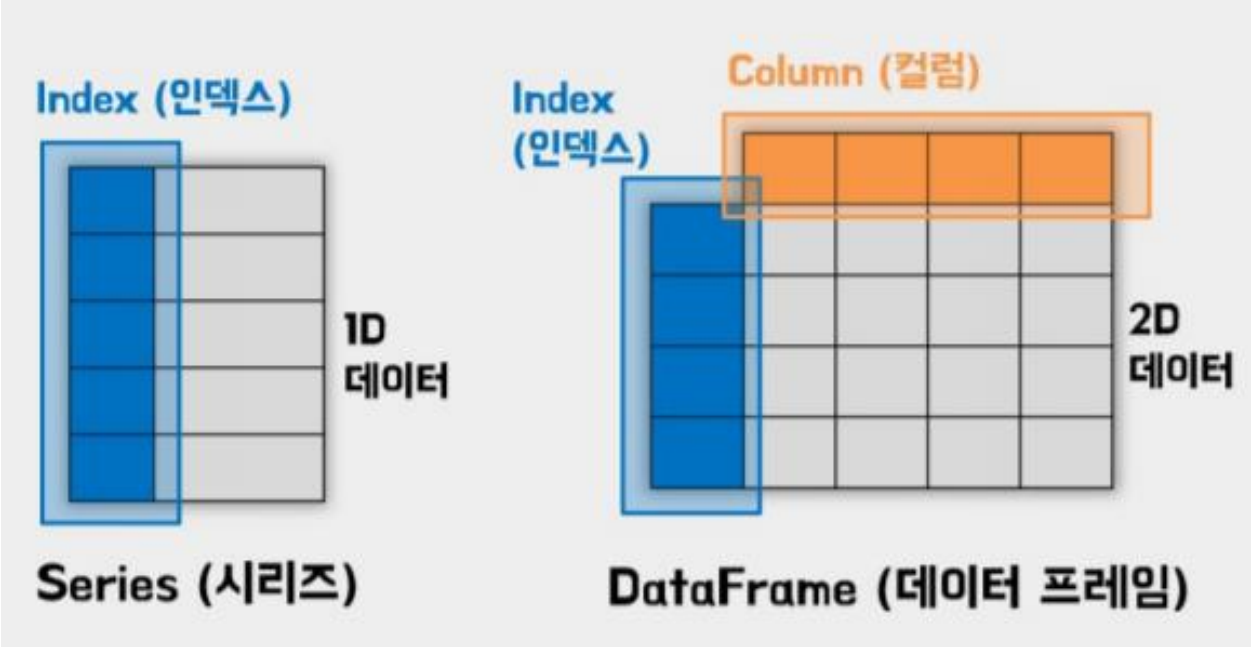
pandas

3. 자료형(클래스)

1) Series : 행과 1개의 열(컬럼)로 구성

2) DataFrame : 행과 열로 구성, 각 열은 Series로 구성

3) Index : 행이나 열을 식별하는 데 사용되는 정수 또는 문자열



Pandas 사용하기

- 라이브러리 임포트

  import pandas as pd

  → pandas 라이브러리를 import하고 앞으로 pd라는 이름으로 부른다.

DataFrame 자료 구조 생성

- read_csv() 함수를 이용한 csv 파일 처리

  df = pd.read_csv(file_path)

	Date	Channel	Age_Group	Gender	Impressions	Clicks	Cost	Conversions	Revenue
0	2025-05-02	Naver	20s	Female	14150	168	297681	24	430532
1	2025-03-03	Facebook	40s	Female	14209	471	75477	2	587067
2	2025-03-27	Facebook	40s	Male	13478	145	204028	22	1744435
3	2025-02-10	Google	30s	Male	13760	645	113312	48	717465
4	2025-06-29	Google	30s	Female	17344	374	288922	3	328810
...	...	...	...	...	...	...	...	...	...
4995	2025-06-11	Naver	20s	Male	18092	298	243651	19	639552
4996	2025-05-18	Facebook	30s	Male	19434	752	73216	37	403496
4997	2025-03-31	Naver	20s	Female	11433	443	206850	22	719760
4998	2025-03-20	Google	40s	Female	19571	898	56744	30	1262333
4999	2025-01-16	Naver	50s+	Male	7615	522	281750	7	1861356

데이터 소개

- 가상의 이커머스(E-commerce) 회사가 2025년 1월 1일부터 6월 30일까지 집행한 디지털 광고의 일별 성과를 기록한 데이터
- 컬럼별 상세 설명
  1) Date: 광고 성과가 기록된 날짜
  2) Channel: 광고가 집행된 마케팅 채널 또는 유입 경로
  3) Age_Group: 해당 광고가 타겟으로 한 고객의 연령대 그룹
  4) Gender: 해당 광고가 타겟으로 한 고객의 성별
  5) Impressions: 광고가 잠재 고객에게 노출된 총 횟수
  6) Clicks: 노출된 광고를 고객이 클릭한 총 횟수
  7) Cost: 해당 광고를 집행하는 데 지출된 총 비용(원)
  8) Conversions: 광고를 통해 발생한 실제 목표 달성 횟수입니다. (예: 제품 구매, 회원 가입, 앱 설치 등)
  9) Revenue: 발생한 전환(Conversions)으로부터 얻은 총 매출액(원)

DataFrame 분해: DataFrame = 행 인덱스 + 컬럼 인덱스 + 값

1. DataFrame 행 인덱스: df.index

2. DataFrame 컬럼 인덱스: df.columns

3. DataFrame 값: df.values →넘파이 2차원 배열

인덱싱(indexing)

- 개념 : 인덱스(index)를 이용하여 데이터프레임으로부터 특정한 데이터를 추출하는 것

- 특정 컬럼 인덱싱

  1) df['컬럼명']

  2) 특정 컬럼 1개에 대한 인덱싱의 결과 → Series 자료 구조 생성

- fancy 인덱싱: 여러 개의 컬럼을 동시에 인덱싱(추출) 하는 것

  : df[['컬럼 이름1', '컬럼 이름2',...]]

- 특정 행과 컬럼 인덱싱

  1) df.loc[행 인덱스, '컬럼 이름']

  2) df.loc[[행 인덱스1, 행 인덱스2], ['컬럼이름1', '컬럼이름2']]

새로운 컬럼 추가(파생 변수 생성)

- CTR (Click-Through Rate): 클릭률 컬럼 추가

```
df['CTR'] = df['Clicks'] / df['Impressions'] * 100
```

	Date	Channel	Age_Group	Gender	Impressions	Clicks	Cost	Conversions	Revenue	CTR
0	2025-05-02	Naver	20s	Female	14150	168	297681	24	430532	1.187279
1	2025-03-03	Facebook	40s	Female	14209	471	75477	2	587067	3.314800
2	2025-03-27	Facebook	40s	Male	13478	145	204028	22	1744435	1.075827
3	2025-02-10	Google	30s	Male	13760	645	113312	48	717465	4.687500
4	2025-06-29	Google	30s	Female	17344	374	288922	3	328810	2.156365

value_counts()

- 개념: 컬럼(column) → 범주형 데이터 → 항목별 개수(빈도수)를 알려주는 함수
- 구현 방법
  1) df['col'].value_counts() 형태로 사용
  2) 예시: Channel 컬럼의 항목별 빈도수

```
### Channel 컬럼의 항목별 빈도수
channel_counts = df['Channel'].value_counts()
print(f'Channel 컬럼의 항목별 빈도수: ₩n{channel_counts}')
```

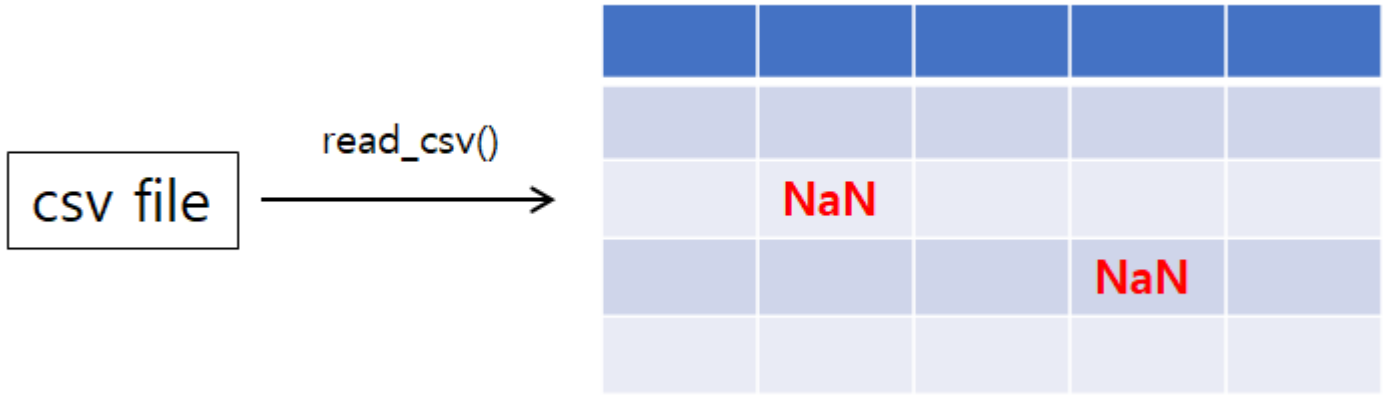
```
Channel 컬럼의 항목별 빈도수:
Channel
Google      1506
Facebook    1479
Naver       1023
Instagram    537
Organic      455
Name: count, dtype: int64
```

08. 정형 데이터 분석 및 시각화

데이터 전처리(preprocessing)

1. 누락 데이터 처리하기

1) 누락 데이터(missing value) : 값이 누락된 데이터



데이터 전처리(preprocessing)

1. 누락 데이터 처리하기

2) 누락된 데이터를 표현하는 용어

① NaN, null, na

② NaN : pandas의 read 함수(read_csv 등)를 이용해서 파일을 DataFrame이나 Series로 변환할 때, 누락 데이터를 표현하는 수단

③ null, na : 함수의 이름으로 사용

3) 누락된 데이터의 개수 확인 함수 : df.isnull().sum()

① isnull() 함수 : 누락 여부를 확인해서 bool 형식으로 전달하는 함수

② sum() : 배열 값의 총합을 구하는 메서드

데이터 전처리(preprocessing)

1. 누락 데이터 처리하기

4) 누락된 데이터 삭제하기: df.dropna(ignore_index)

① 행 또는 열에 누락 데이터(missing value, NaN)가 1개만 있어도 해당 행 제거

② ignore_index=True : 행 인덱스 자동 수정

데이터 전처리(preprocessing)

1. 누락 데이터 처리하기

5) 누락된 데이터 바꾸기: 적당한 데이터 삽입

① df_cleaned = df.fillna("내용 없음")

② df_cleaned = df.fillna(0)

데이터 전처리(preprocessing)

2. 중복되는 데이터를 그룹별로 모아서 처리하기

1) groupby()

① 중복된 데이터를 그룹별로 분할

② 분할된 각 그룹에 대하여 별도로 데이터를 처리하거나 그룹별 통계량을 확인하고자 할 때 사용

🏢 지점별 사원 실적 데이터 예시

사원ID	이름	지점	실적 (만원)
101	김철수	**서울**	500
102	이영희	**서울**	300
103	박민수	**부산**	450
104	최제이	**서울**	400
105	정팀장	**부산**	350
106	홍길동	**서울**	600

📊 지점별 실적 합계 정렬 결과 (내림차순)

지점	실적 합계 (만원)	실적 평균 (만원)
서울	**1,800**	450
부산	**800**	400

데이터 전처리(preprocessing)

2. 중복되는 데이터를 그룹별로 모아서 처리하기

2) groupby(by).func()

① by=[co1, ...] : 중복 여부를 확인하여 그룹별로 묶을 컬럼 지정

② func(함수) : 그룹별로 묶은 데이터에 대한 특정 함수 적용 예) sum(), mean() ...

"어떤 채널의 평균 매출이 가장 높은가?" → 채널별 Revenue 컬럼의 평균 값 계산

```
channel_rev = df.groupby(by='Channel')['Revenue'].mean()
```

데이터 전처리(preprocessing)

2. 중복되는 데이터를 그룹별로 모아서 처리하기

3) groupby(by).agg(새로운_컬럼명=('원본 컬럼명', '집계 함수'))

: 기능 : 각 그룹별로 다양한 함수를 동시에 적용

"각 광고 채널별로 지출한 비용의 총합과 평균, 매출액의 총합과 평균은 어떻게 되는가?"

```
channel_summary = df.groupby('Channel').agg(
    total_cost=('Cost', 'sum'),
    average_cost=('Cost', 'mean'),
    total_revenue=('Revenue', 'sum'),
    average_revenue=('Revenue', 'mean')
)
```

데이터 전처리 (preprocessing)

3. 이상치(outlier) 처리하기

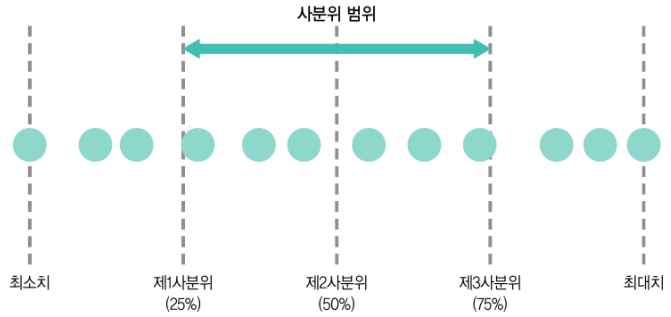
1) 이상치 : 수집된 데이터 중에서 아주 작은 값이나 매우 큰 값

2) 이상치 측정 : IQR 이용

① 사분위수 : 순서대로 정렬된 데이터를 4구간으로 균등하게 분할 할 때 기준이 되는 수

② 백분위수 : 순서대로 정렬된 데이터를 균등하게 100 등분하는 경우 기준이 되는 수

③ IQR(Interquartile range) : 사분위수의 75% 지점의 값과 25% 지점의 값 차이(Q3 - Q1)



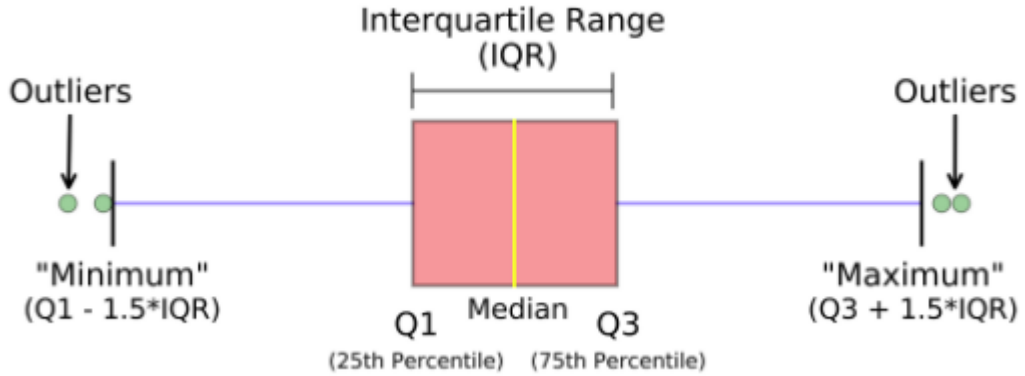
④ 정상 범위 데이터의 최소값(min) = Q1 – (1.5 * IQR)

⑤ 정상 범위 데이터의 최대값(max) = Q3 + (1.5 * IQR)

데이터 전처리 (preprocessing)

3. 이상치(outlier) 처리하기

⑤ 이상치 : min, max를 벗어난 값



4) 분위수(quantile) 구하기: quantile(q) 함수

① q : 원하는 백분위수 설정

② df.quantile(q) / df['col'].quantile(q)

데이터 전처리 (preprocessing)

3. 이상치(outlier) 처리하기

3) 이상치 처리 방법

① 제거

- 전체 수치형 컬럼에 대한 Q1, Q3, IQR 한 번에 계산
- 각 컬럼별 정상 범위 최소값/최대값 계산
- 데이터프레임 전체에 대해 논리 연산자 적용, 이상치 여부 판단 → 블리언 DataFrame 생성
- 어떤 컬럼에서든 이상치가 하나라도 있는 행 식별
- 블리언 인덱싱을 통한 이상치가 없는 행만 필터링 후 인덱스 재설정

② 대체

- 정상 범위 최소값 미만은 정상 범위 최소값으로, 정상 범위 최대값 초과는 정상 범위 최소값으로 일괄 대체
- clip 함수 사용: df_clipped = df.clip(lower=min_val, upper=max_val, axis=1)

데이터 전처리 (preprocessing)

3. 이상치(outlier) 처리하기 실습: 기상청 날씨 데이터 이용

1) 비교 연산자

① 두 변수의 크기를 비교할 수 있는 도구

② 결과 값 : 참(True), 거짓(False)

종류	설명
>	왼쪽 값이 크다
<	왼쪽 값이 작다
>=	왼쪽 값이 크거나 같다
<=	왼쪽 값이 작거나 같다
==	값이 같다
!=	값이 같지 않다

2) 논리 연산자

① 두 가지 이상의 조건(비교 연산)을 동시에 처리할 때 사용하는 도구

② 종류 : 논리곱 , 논리합, ~논리부정

논리 연산자	문법	설명
and	a and b	AND(논리곱), 양쪽 모두 참일 때 참
or	a or b	OR((논리합), 양쪽 중 한쪽만 참이라도 참
not	not x	NOT(논리부정), 참과 거짓을 뒤집음

데이터 탐색

1. 통계로 요약하기

1) 요약(기술) 통계량 구하기

① 요약 통계량

- 데이터 분포의 특징을 나타내는 통계량,
- 데이터 분석의 과정에서 데이터의 전체적인 특징과 분포를 빠르게 파악하기 위해 사용
- 구현 방법: df.describe()

② 숫자(Numeric) 데이터의 경우 (기본 동작)

- count: 데이터 개수 (NaN 값 제외)
- mean: 평균값
- std: 표준편차 (데이터가 퍼져있는 정도)
- min: 최솟값
- 25%: 1사분위수 (하위 25% 지점의 값)
- 50%: 중앙값 (median, 2사분위수)
- 75%: 3사분위수 (하위 75% 지점의 값)
- max: 최댓값

데이터 탐색

1. 통계로 요약하기

2) 요약 통계량으로부터 평균 구하기

- 행: mean, 컬럼: Cost → “광고 캠페인 하나에 평균적으로 얼마의 비용을 쓰고 있는가?”
- df.loc[행, 열] 사용

데이터 탐색

1. 통계로 요약하기

3) 요약 통계량으로부터 최솟값, 최대값 구하기

- 행: min, 컬럼: Revenue → "매출의 최소는 얼마인가?"
- 행: max, 컬럼: Revenue → "매출의 최대는 얼마인가?"
- df.loc[행, 열] 사용

데이터 탐색

1. 통계로 요약하기

4) 요약 통계량으로부터 분위수(quantile) 구하기

- 행: 50%, 컬럼: Revenue → "Revenue 컬럼의 중간값(중앙값)은 얼마인가?"
- df.loc[행, 열] 사용

데이터 탐색

1. 통계로 요약하기

5) 요약 통계량으로부터 표준편차(standard deviation) 구하기

① 표준편차 : 데이터들이 평균값에서 평균적으로 얼마나 떨어져 있는가?

- 데이터의 '일관성' 또는 변동성을 보여주는 지표
- 표준편차가 작다: 데이터들이 평균값 주변에 옹기종기 모여있다. (성과가 꾸준하고 예측 가능)
- 표준편차가 크다: 데이터들이 평균값에서 멀리, 넓게 흩어져 있다. (성과가 들쭉날쭉하고 예측하기 힘듦)

② 요약 통계량으로부터 결과 가져오기

- 행: std, 컬럼: Revenue → "Revenue 컬럼의 표준편차는 얼마인가?"
- df.loc[행, 열] 사용

데이터 탐색

2. 상관관계 분석(Correlation Analysis)

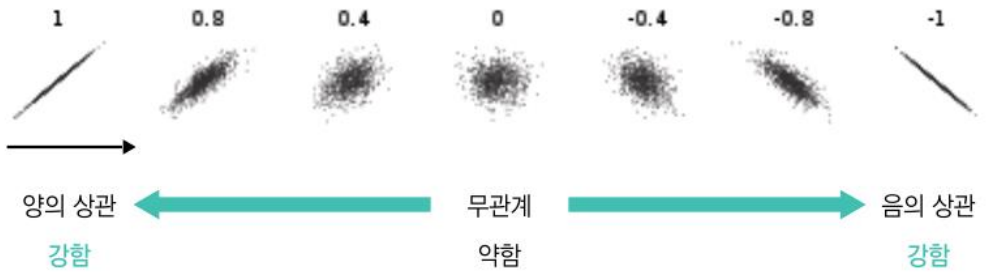
1) 개념 : 두 변수 사이에 연관성의 존재 여부와 연관성의 강도를 측정하여 분석하는 방법

예) 기업에서 광고비 지출이 매출액의 증가에 어느 정도의 영향이 있는 지를 파악

2) 상관관계의 종류

종류	설명
양(+)의 상관관계	• 한 변수의 값이 증가할 때 다른 변수의 값도 증가한다 • 강도에 따라 강한 양의 상관관계, 약한 양의 상관관계가 있다
음(-)의 상관관계	• 한 변수의 값이 증가할 때 다른 변수의 값은 감소한다 • 강도에 따라 강한 음의 상관관계, 약한 음의 상관관계가 있다
상관관계가 없음	한 변수의 값의 변화에 무관하게 다른 변수의 값이 변화한다

3) 상관계수(피어슨 상관계수) : 두 변수 간에 존재하는 선형 관계의 정도를 수량화, +1 ~ -1



데이터 탐색

2. 상관관계 분석 (Correlation Analysis)

4) 구현 방법

- df.corr(numeric_only=True) : 변수들 간의 상관 계수를 표현한 데이터프레임 생성 → 상관 행렬

	Impressions	Clicks	Cost	Conversions	Revenue
Impressions	1.000000	0.104543	0.009052	-0.000656	0.011986
Clicks	0.104543	1.000000	0.001509	0.046552	0.015925
Cost	0.009052	0.001509	1.000000	-0.005274	0.009770
Conversions	-0.000656	0.046552	-0.005274	1.000000	-0.016981
Revenue	0.011986	0.015925	0.009770	-0.016981	1.000000

데이터 탐색

3. 탐색적 데이터 분석(EDA, Exploratory Data Analysis)

1) 개념: 시각화를 통한 데이터에 대한 이해(특징과 패턴 파악)

2) 시각화 (visualization)

① 데이터를 도표나 차트 등으로 정리하는 것

② 시각화를 통해 데이터의 특징을 쉽게 파악 할 수 있다

③ 분석 결과를 상대방에게 효과적으로 전달 가능하다

데이터 탐색

3. 탐색적 데이터 분석(EDA, Exploratory Data Analysis)

3) python 시각화 라이브러리

① matplotlib

② seaborn

③ plotly

4) 주로 사용되는 시각화 도구

① 직선 그래프

② 막대 그래프

③ 원 그래프

④ 히스토그램

⑤ 박스 플롯

⑥ 산점도

⑦ 히트맵

데이터 탐색

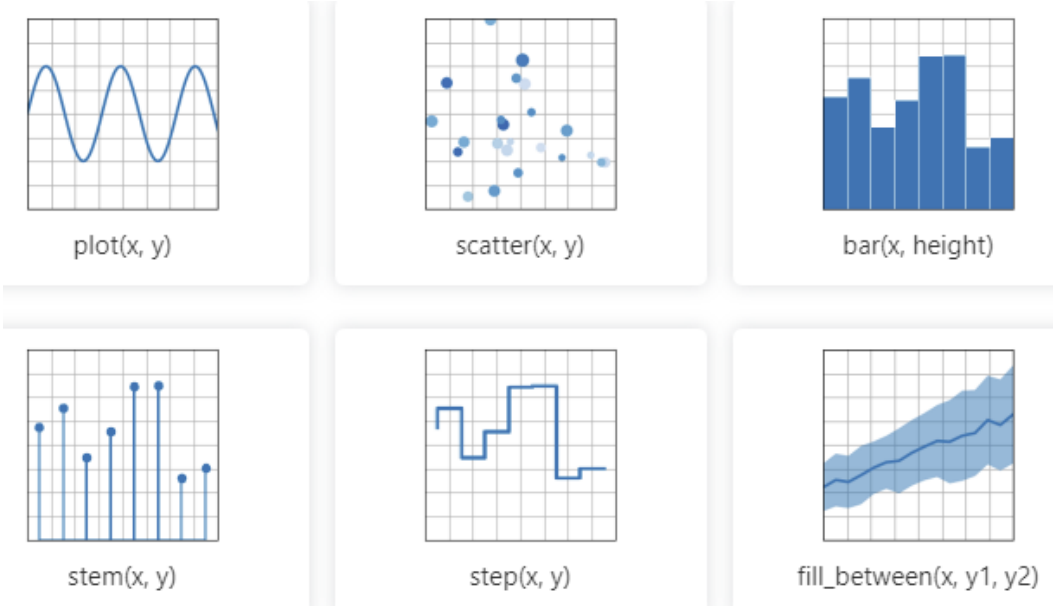
3. 탐색적 데이터 분석(EDA, Exploratory Data Analysis)

1) python 시각화 라이브러리: matplotlib

① Python에서 시각화 할 수 있는 대표적인 라이브러리

② matplotlib 사용하기

```
#  import matplotlib.pyplot as plt
```



데이터 탐색

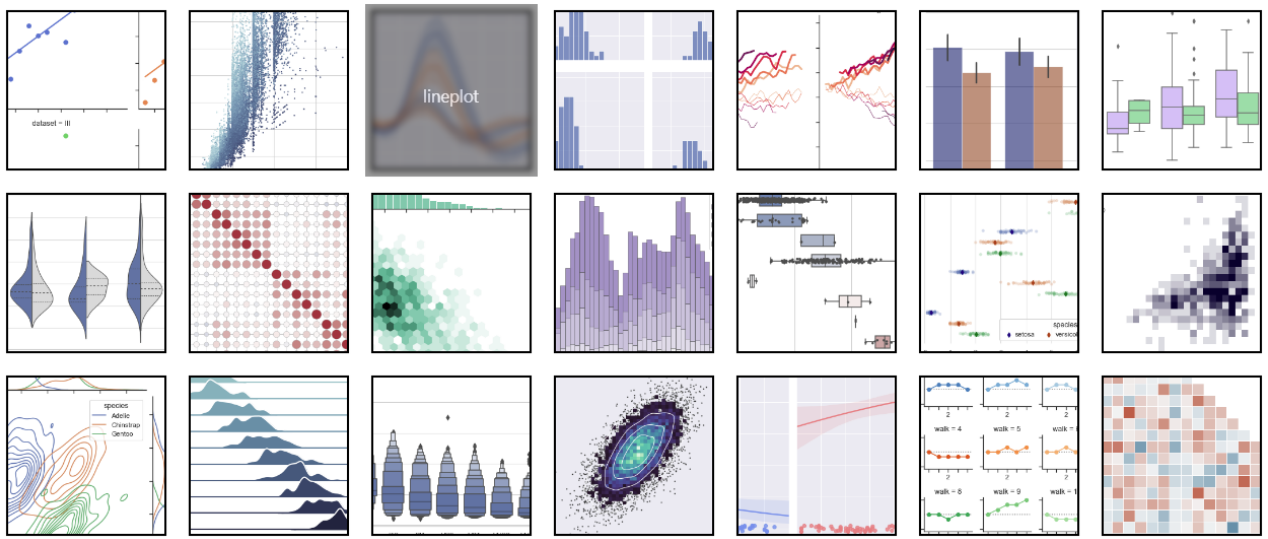
3. 탐색적 데이터 분석(EDA, Exploratory Data Analysis)

2) python 시각화 라이브러리: seaborn

① matplotlib을 기반, 통계 데이터 시각화 라이브러리

② seaborn 사용하기

```
#  import seaborn as sns
```



데이터 탐색

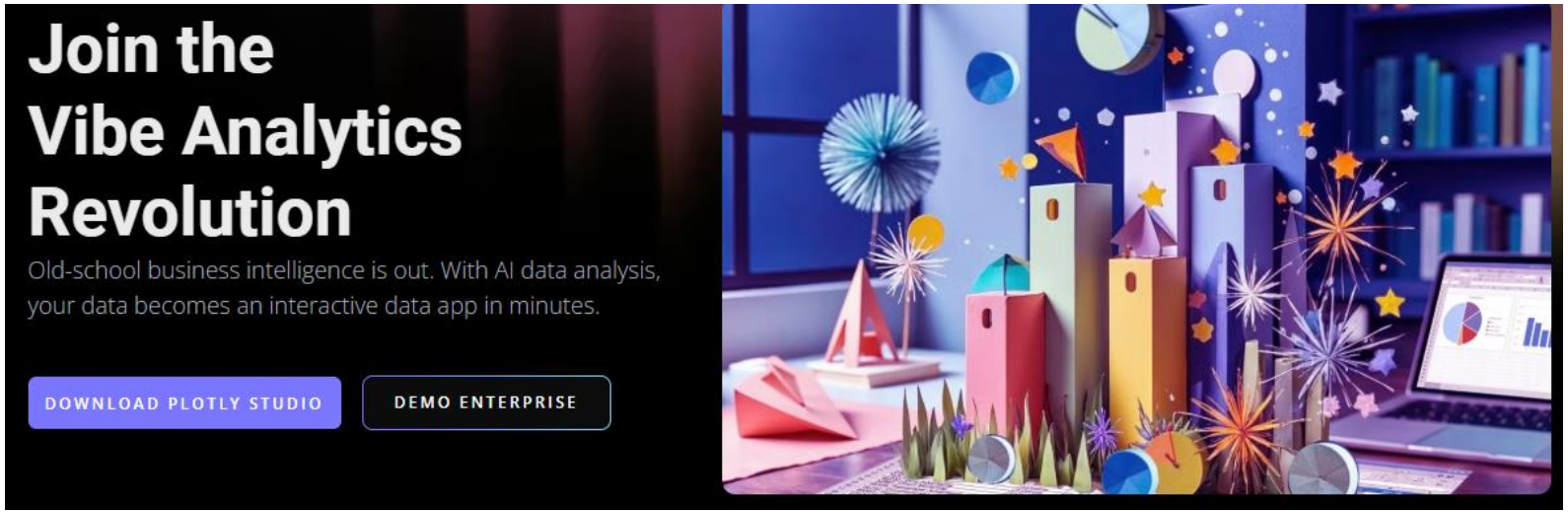
3. 탐색적 데이터 분석(EDA, Exploratory Data Analysis)

3) python 시각화 라이브러리: plotly

① 개념: 웹 브라우저 기반의 인터랙티브 시각화 라이브러리, 마우스 오버 시 데이터 값 표시(툴팁), 드래그를 통한 확대/축소, 범례 클릭을 통한 필터링 등 강력한 기능을 제공

② plotly 사용하기

```
# import plotly.express as px
```



데이터 탐색

3. 탐색적 데이터 분석(EDA, Exploratory Data Analysis)

4) python 시각화 라이브러리 plotly을 이용한 시각화

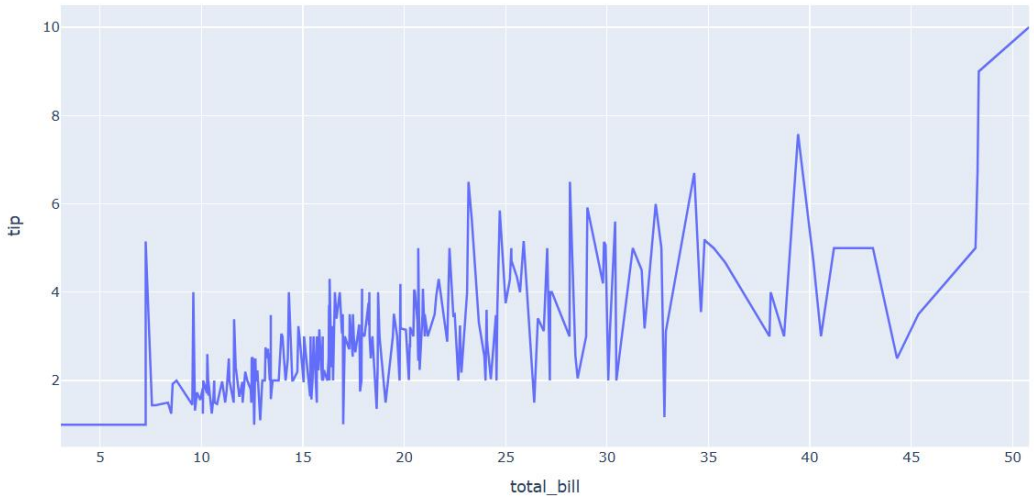
(1) 직선 그래프

① 개념: 두 개 이상의 데이터 포인트를 직선으로 연결하여 연속적인 변화와 추세를 시각적으로 표현하는 그래프

② 구현 방법: px.line(data_frame, x, y, color)

③ 하이퍼파라미터

- data_frame: 데이터프레임
- x, y: 시각화 할 컬럼
- color: 새로운 변수를 추가하여 색상으로 구분



데이터 탐색

3. 탐색적 데이터 분석(EDA, Exploratory Data Analysis)

4) python 시각화 라이브러리 plotly을 이용한 시각화

(2) 막대 그래프

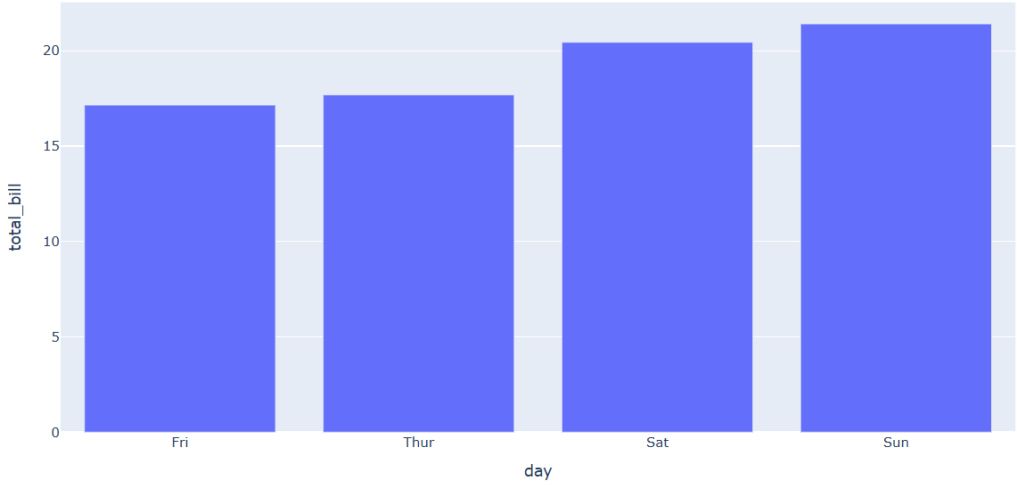
① 개념: 범주형 컬럼에 대해 범주별 수치 데이터의 크기(총합, 평균 등)를 직사각형 막대로 표현하는 그래프

② 구현 방법

- 수직 막대: px.bar(data_frame, x, y)
- 수평 막대: px.bar(data_frame, x, y, orientation='h')

③ 하이퍼파라미터

- data_frame : 데이터프레임
- x, y:
  - 수직형: x가 범주형 컬럼, y가 수치형 컬럼
  - 수평형: x가 수치형 컬럼, y가 범주형 컬럼
- orientation: 'v' (수직, 기본값), 'h' (수평)



데이터 탐색

3. 탐색적 데이터 분석(EDA, Exploratory Data Analysis)

4) python 시각화 라이브러리 plotly을 이용한 시각화

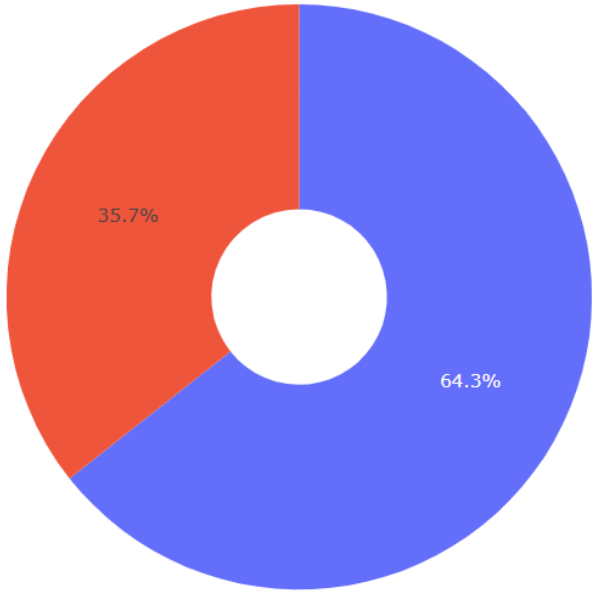
(3) 원 그래프

① 개념 : 범주형 컬럼에서 전체에 대한 각 범주의 비율을 시각적으로 나타내며, 부분과 전체의 관계를 파악하기 용이한 그래프

② 구현 방법: px.pie(data_frame, names, values, hole)

③ 하이퍼파라미터

- data_frame: 데이터프레임
- names: 범주형 컬럼 (원의 조각 이름)
- values: 수치형 컬럼 (조각의 크기)
- hole: 0~1 사이 값을 설정하여 도넛 모양으로 변경



데이터 탐색

3. 탐색적 데이터 분석(EDA, Exploratory Data Analysis)

4) python 시각화 라이브러리 plotly을 이용한 시각화

(4) 히스토그램(Histogram)

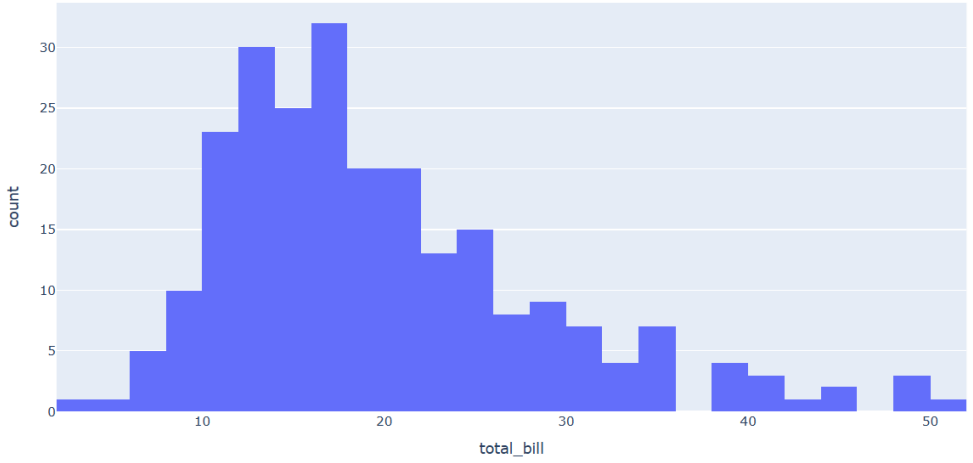
① 개념: 수치형 데이터의 분포를 구간(bin)별로 나누어 구간별 누적 빈도를 직사각형의 막대로 표현하는 그래프

② 구현 방법

- 수직형: px.histogram(data_frame, x, y=None, nbins, color)
- 수평형: px.histogram(data_frame, x=None, y, color)

③ 하이퍼파라미터

- data_frame: 데이터프레임
- x, y:
  - 수직형: x가 수치형 컬럼, y=None
  - 수평형: x =None, y가 수치형 컬럼
- nbins: 빈(bin)의 개수 설정 (미지정 시 최적값 자동 계산)
- color: 새로운 변수를 추가하여 색상으로 구분



데이터 탐색

3. 탐색적 데이터 분석(EDA, Exploratory Data Analysis)

4) python 시각화 라이브러리 plotly을 이용한 시각화

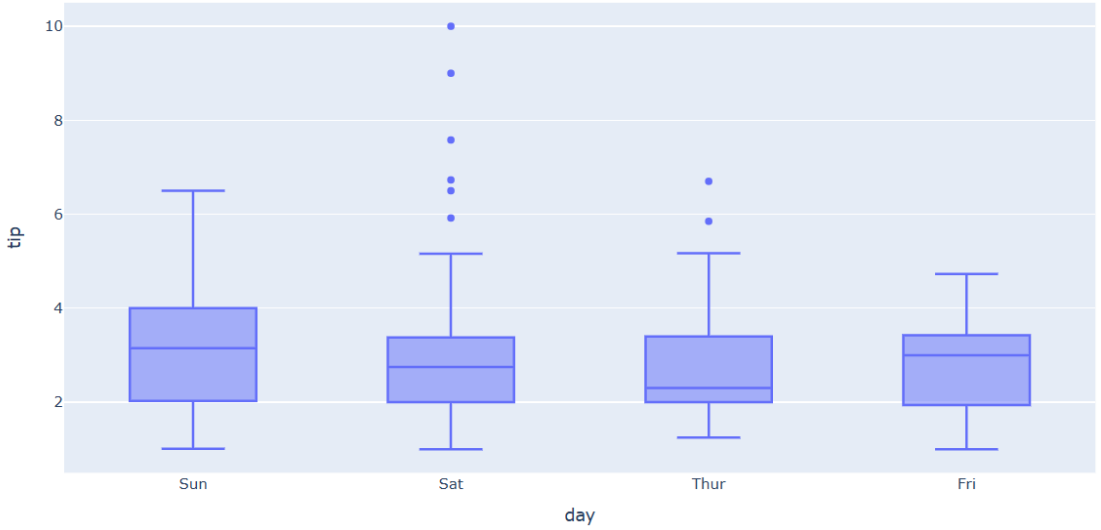
(5) 박스 플롯(Box plot)

① 개념: 수치형 컬럼에 대해서 최소값(min), 1사분위수(Q1), 2사분위수(Q2, median), 3사분위수(Q3), 최대값(max)을 이용하여 데이터의 분포를 시각적으로 요약하고 이상치를 식별하는데 유용한 그래프

② 구현 방법: px.box(data_frame, x, y, color)

③ 하이퍼파라미터

- data_frame: 데이터프레임
- x, y:
  - 수직형: x가 범주형 컬럼, y가 수치형 컬럼
  - 수평형: x가 수치형 컬럼, y가 범주형 컬럼
- color: 새로운 변수를 추가하여 색상으로 구분



데이터 탐색

3. 탐색적 데이터 분석(EDA, Exploratory Data Analysis)

4) python 시각화 라이브러리 plotly을 이용한 시각화

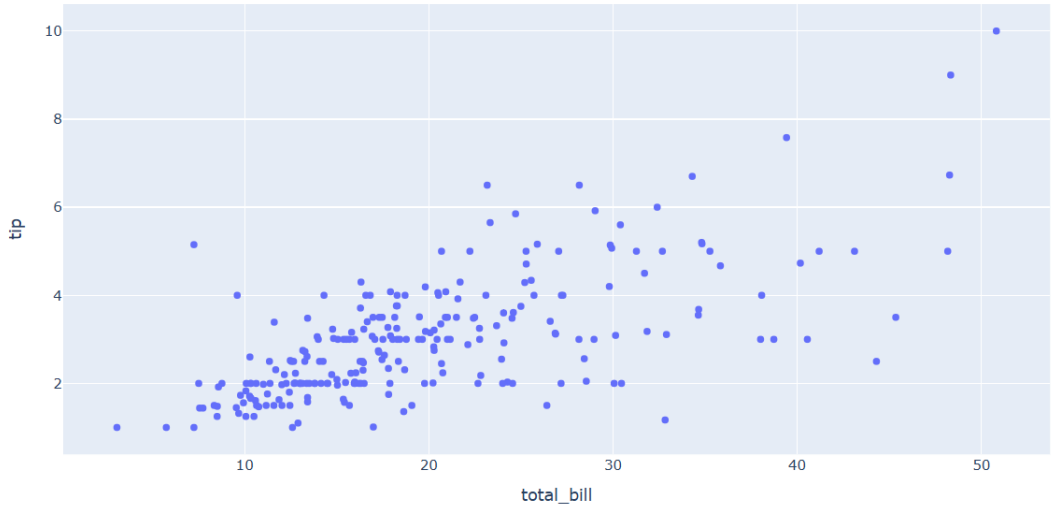
(6) 산점도(Scatterplot)

① 개념: 두 변수의 상관 관계를 직교 좌표계의 평면에 점으로 표현하는 그래프

② 구현 방법: px.scatter(data_frame, x, y, color)

③ 하이퍼파라미터

- data_frame: 데이터프레임
- x, y: 상관 관계를 분석할 두 수치형 컬럼
- color: 그룹별 색상 구분



데이터 탐색

3. 탐색적 데이터 분석(EDA, Exploratory Data Analysis)

4) python 시각화 라이브러리 plotly을 이용한 시각화

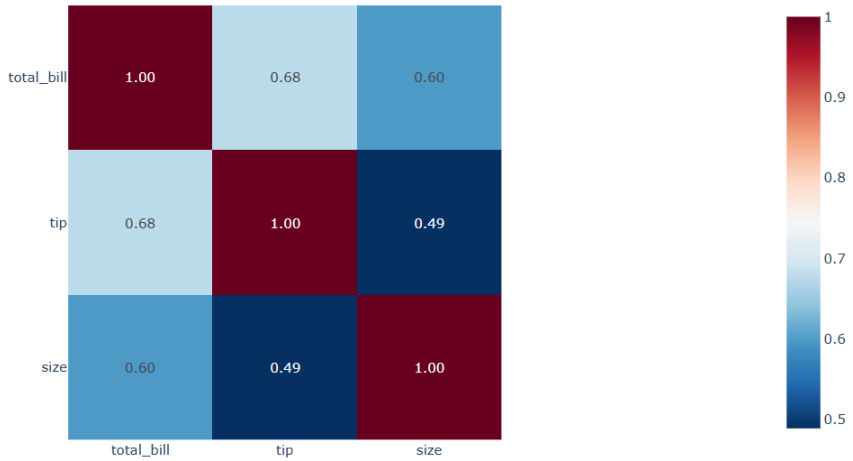
(7) heatmap

① 개념: 변수들 간의 상관 계수 크기를 색상의 농도로 표현하여 변수 간의 관계를 한눈에 파악하는 그래프

② 구현 방법: px.imshow(data_frame, text_auto, color_continuous_scale)

③ 하이퍼파라미터

- data_frame: 상관 행렬(데이터프레임)
- text_auto: 상관계수 수치 형식을 표시 (text_auto='.2f')
- color_continuous_scale: 히트맵의 색상 테마 설정
  (예: 'RdBu_r', 'Viridis')




MISSION:
COMPLETE